

BÀI BÁO CÁO MÔN BIG DATA VÀ ỨNG DỤNG

THIẾT KẾ HỆ THỐNG PHÂN TÁN CHO PHÂN TÍCH LUỒNG VIDEO TOÀN CẦU

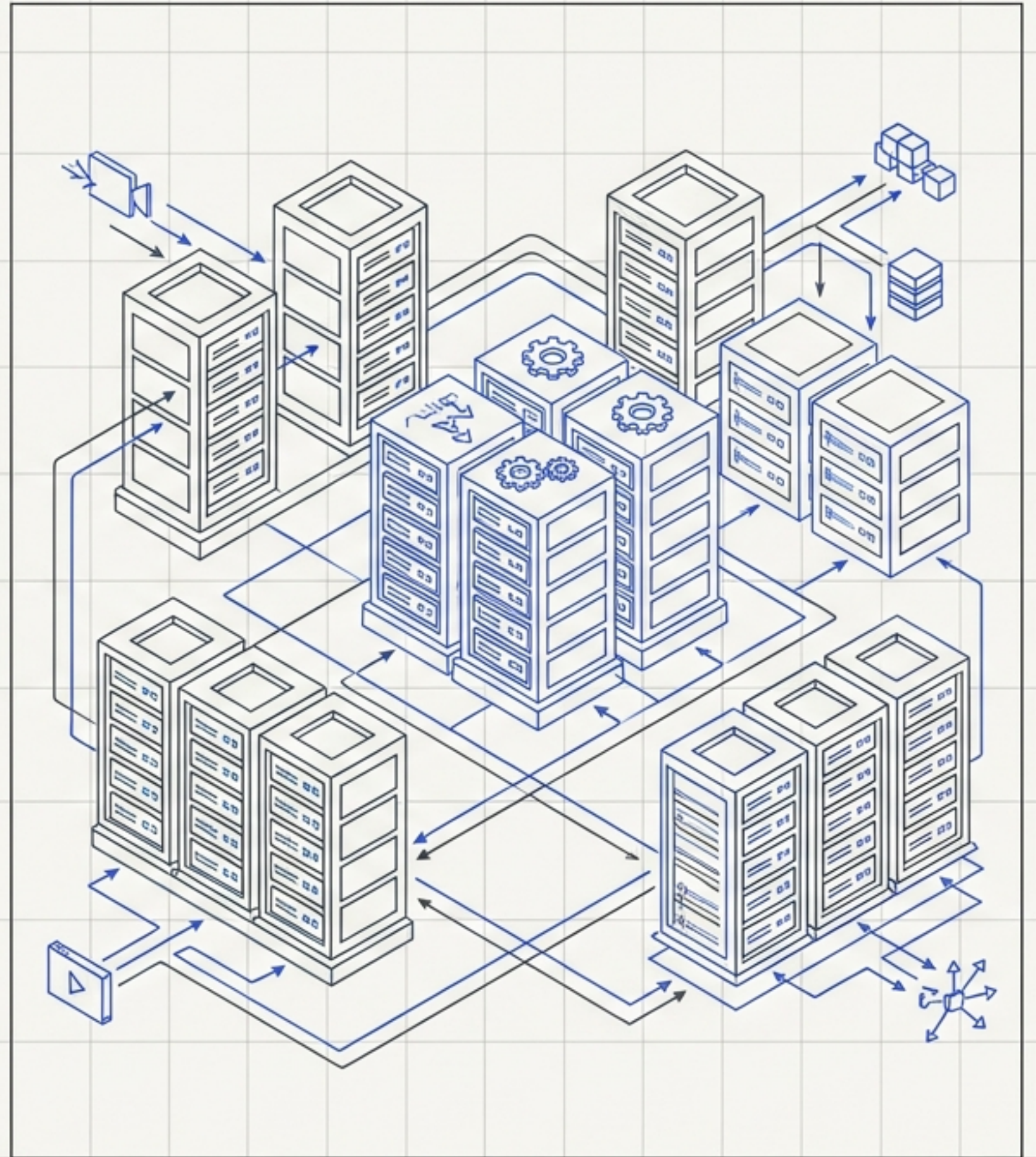
Giải pháp kiến trúc thu thập, xử lý
và lưu trữ dữ liệu thời gian thực

Nhóm 6: Huỳnh Thế Hy (051205009083), Nghiêm Đức Thuận
(060205003756), Phan Trần Nhật Hưng (079205042911), Nguyễn
Khánh Dương (042205000026)

Giảng viên hướng dẫn: Hàn Trung Định

Mã lớp: 010412711004

Viện Đào tạo Chất lượng cao - ĐHGTVT TP.HCM | Tháng 05/2026





Link Dự Án: github.com/Thehy-IT/BDA_Lab_video-streaming-analytics

Tổng quan Dự án



Mục tiêu (Objectives)

Xây dựng kiến trúc phân tán toàn chỉnh cho hệ thống Video Streaming Analytics toàn cầu. Giải quyết bài toán thu thập, xử lý (**micro-batch**), và lưu trữ dữ liệu thời gian thực.



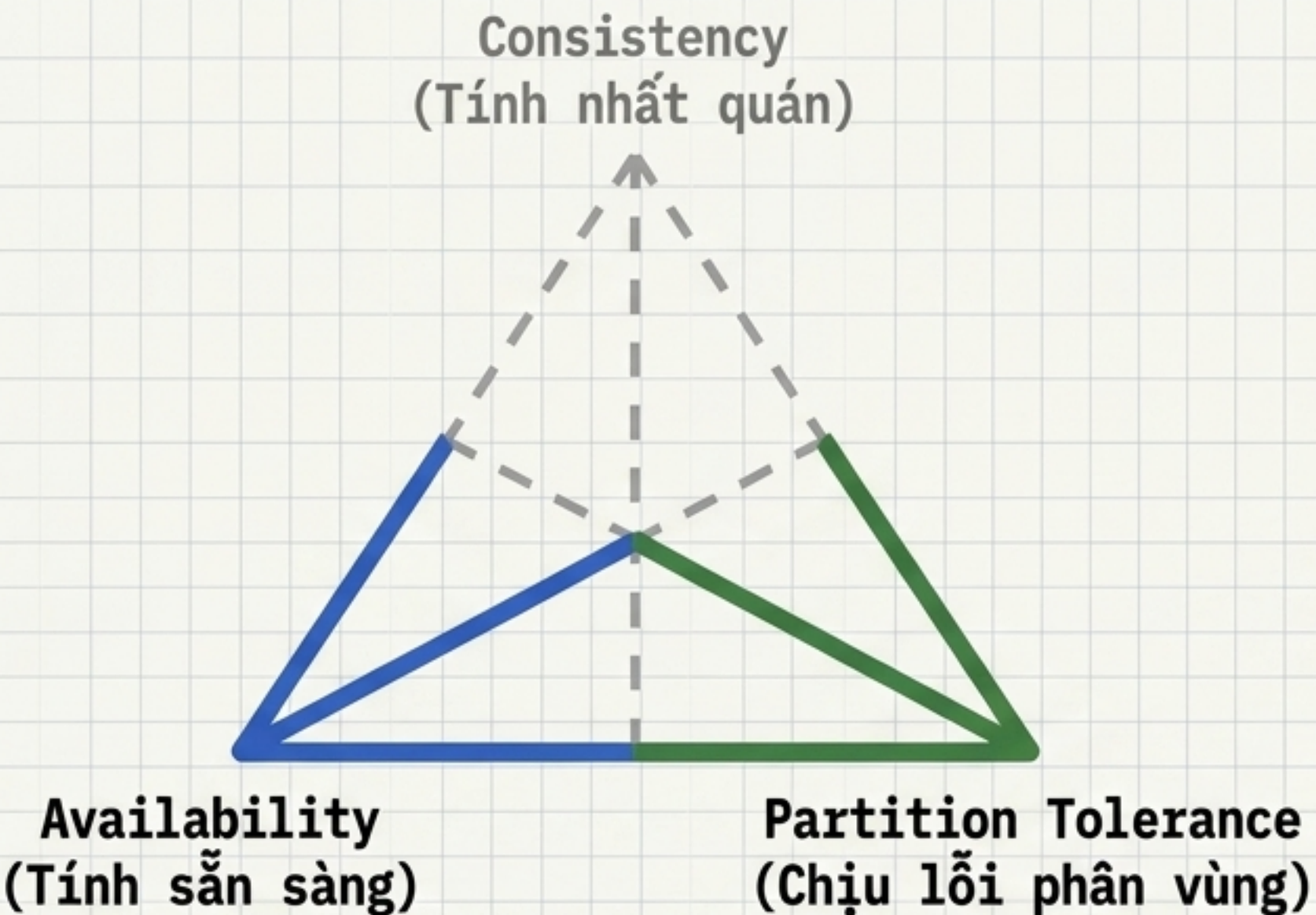
Kiến trúc (Architecture)

Áp dụng mô hình Lambda cải tiến với 4 công nghệ cốt lõi: **Apache Kafka** (Ingestion), **Python** (Processing), **MongoDB** (Storage), và **Streamlit** (Dashboard).



Kết quả (Results)

Triển khai thành công trên 4 máy chủ vật lý. Đảm bảo tính sẵn sàng cao (HA), xử lý **micro-batch** với độ trễ **5-10s**, và tự động phục hồi hoàn toàn sau sự cố (**Chaos Tested**).



Lựa chọn Thiết kế: Ưu tiên hệ thống AP (Availability + Partition Tolerance).

Lý do: Đảm bảo luồng video và trải nghiệm người dùng không bao giờ bị gián đoạn. Chấp nhận một độ trễ nhỏ về tính nhất quán dữ liệu.

Đặc điểm Hệ thống Phân tán

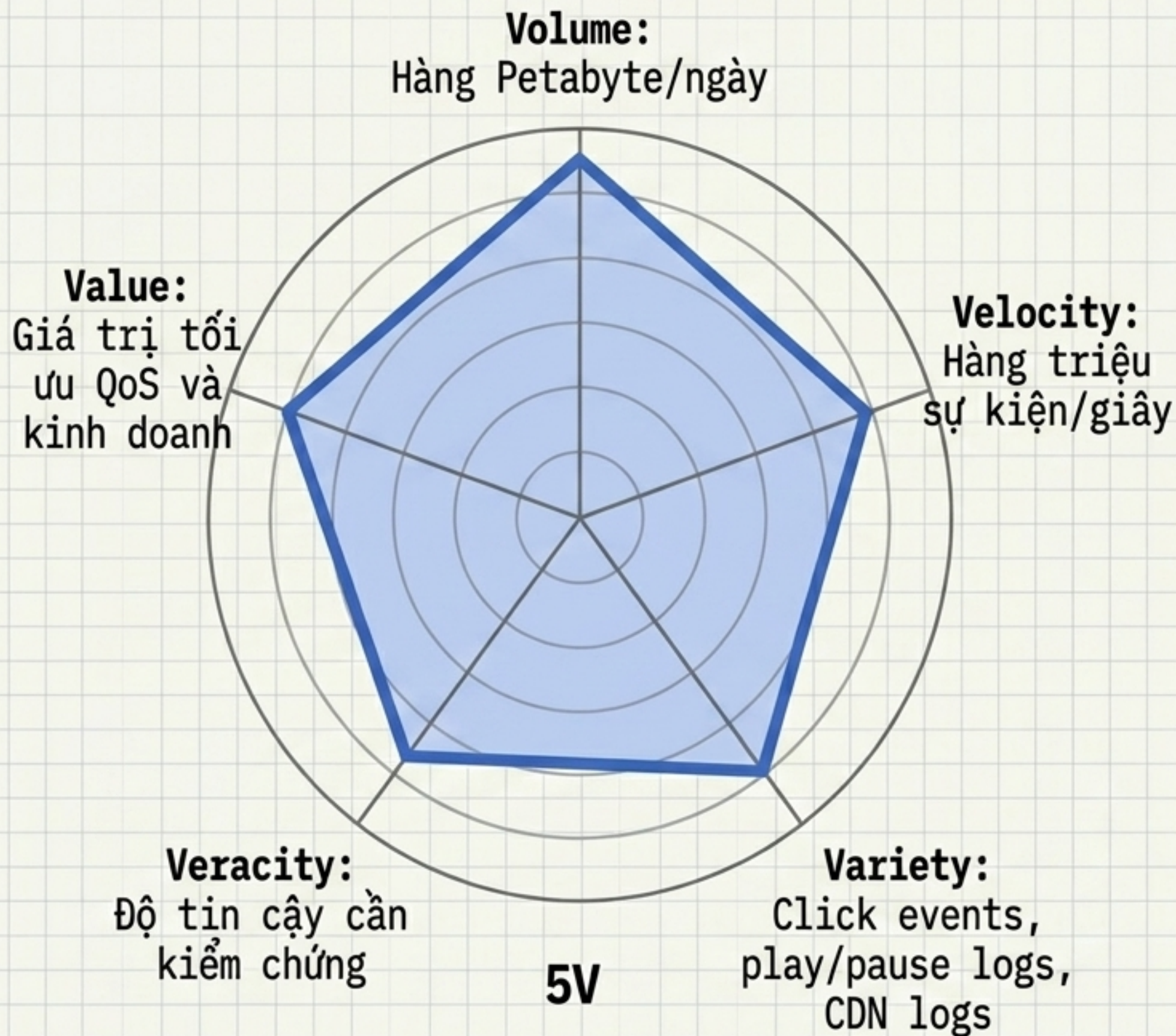
Tính minh bạch (Transparency): Che giấu sự phân tán tài nguyên khỏi người dùng.

Tính mở (Openness): Tuân thủ giao thức chuẩn, dễ mở rộng.

Khả năng mở rộng (Scalability): Hỗ trợ scale-out theo chiều ngang.

Khả năng chịu lỗi (Fault Tolerance): Duy trì hoạt động khi có node sập.

Yêu cầu Phân tích (Analytics Typology)



Real-time Analytics: Đếm lượt xem đồng thời, phát hiện bất thường (buffering, bitrate drops).

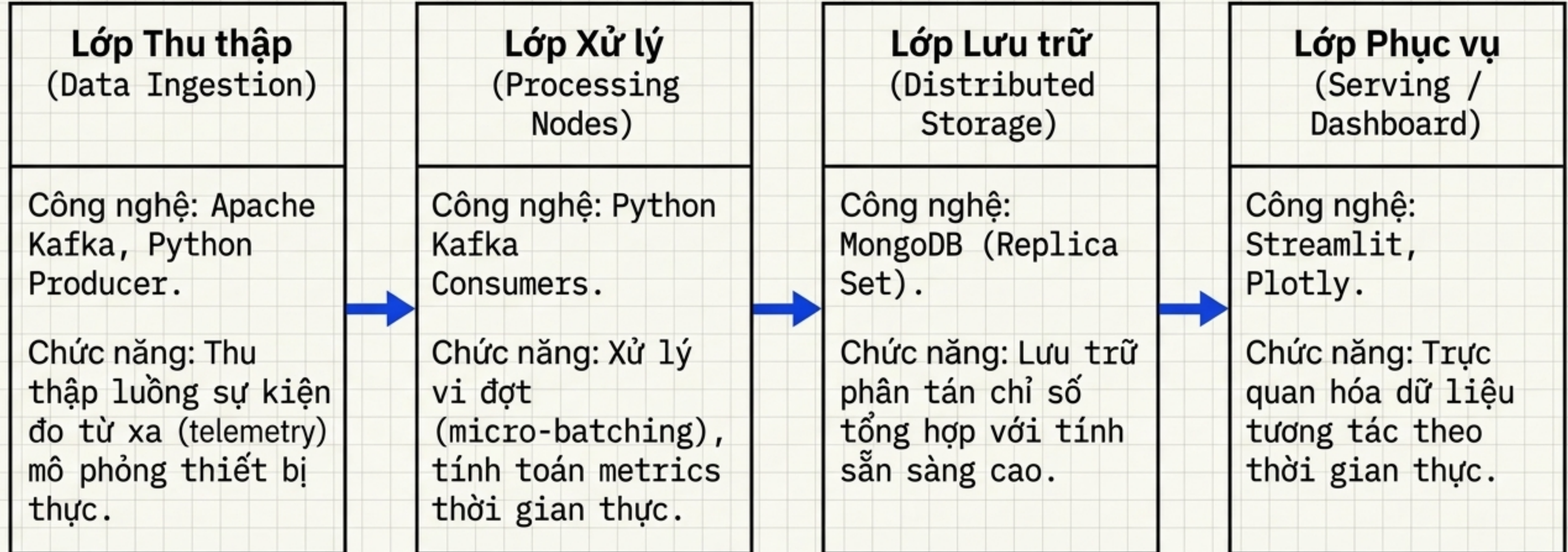
Batch Analytics: Phân tích hành vi theo chu kỳ (ngày/tuần), xây dựng hệ thống gợi ý nội dung.

Geo-distributed Analytics: Phân tích theo vùng địa lý để tối ưu hóa mạng phân phối nội dung (CDN).

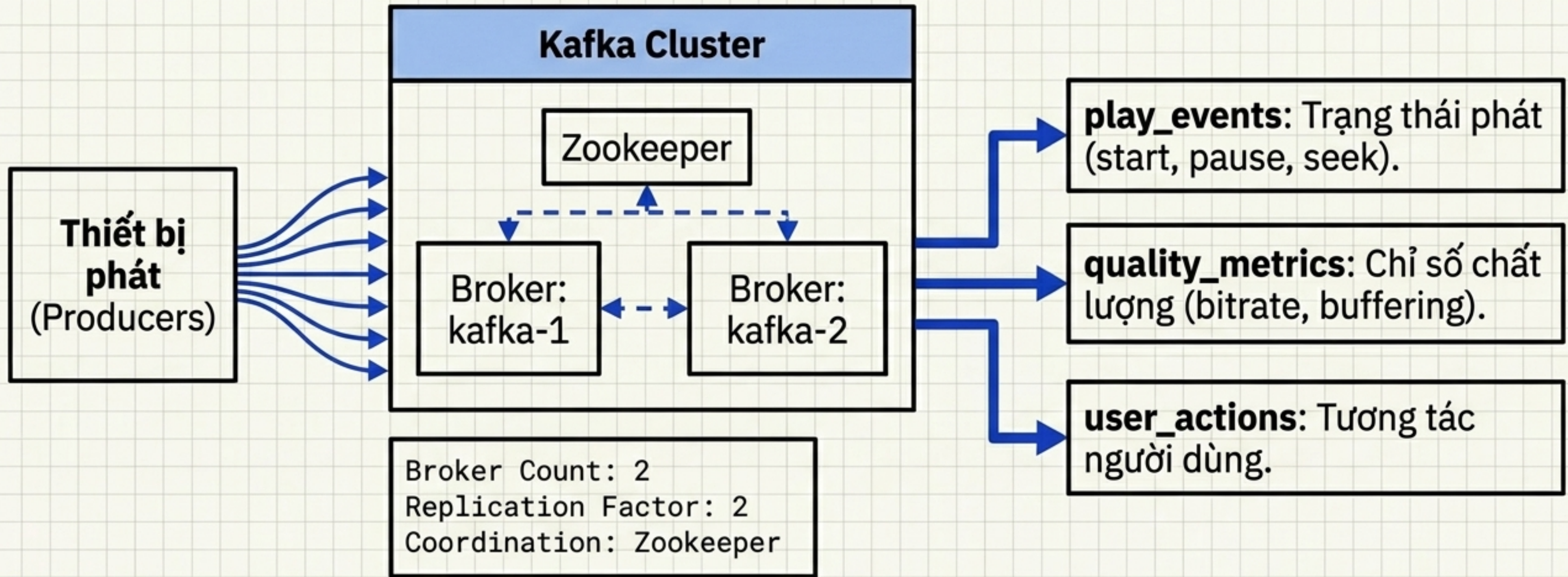
Ma trận Yêu cầu Hệ thống

Yêu cầu Chức năng (Functional)	Yêu cầu Phi chức năng (Non-Functional)
Data Ingestion: Thu thập sự kiện phân tán toàn cầu theo thời gian thực (real-time).	Throughput: > 10 triệu sự kiện/giây.
Storage: Lưu trữ dữ liệu thô và kết quả phân tích, hỗ trợ truy vấn linh hoạt.	Availability: Đạt chuẩn SLA 99.99% (Downtime tối đa < 52 phút/năm).
Processing: Hỗ trợ cả luồng (< 1s độ trễ) và lô (theo giờ).	Durability: Không mất dữ liệu, hệ số nhân bản (Replication Factor) ≥ 3.
Serving: Cung cấp API truy xuất cho các dịch vụ downstream.	Scalability: Mở rộng tuyến tính (Scale-out) khi thêm node.

Kiến trúc Tổng thể (Macro Architecture)



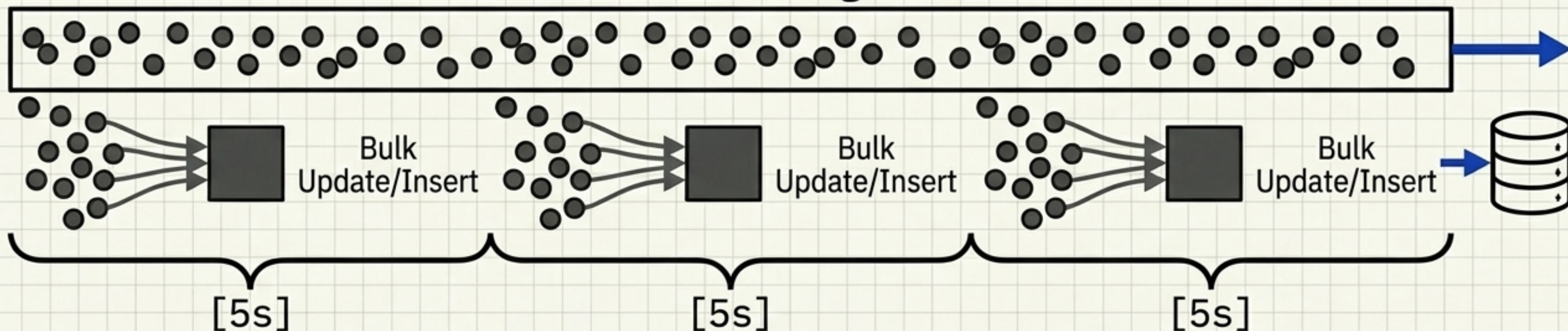
Lớp Thu thập (Ingestion Layer)



Việc chia nhỏ Topic cho phép **Lớp Xử lý (Processing Layer)** tiêu thụ và **tính toán** dữ liệu **hoàn toàn song song** và **độc lập**.

Cơ chế Tumbling Window & Processing

Cơ chế Tumbling Window



Micro-batching (5s Window)

Thay vì ghi từng bản ghi đơn lẻ, hệ thống gom dữ liệu trong cửa sổ 5 giây để thực hiện **Bulk Update/Insert**, giảm tải triệt để cho **MongoDB**.

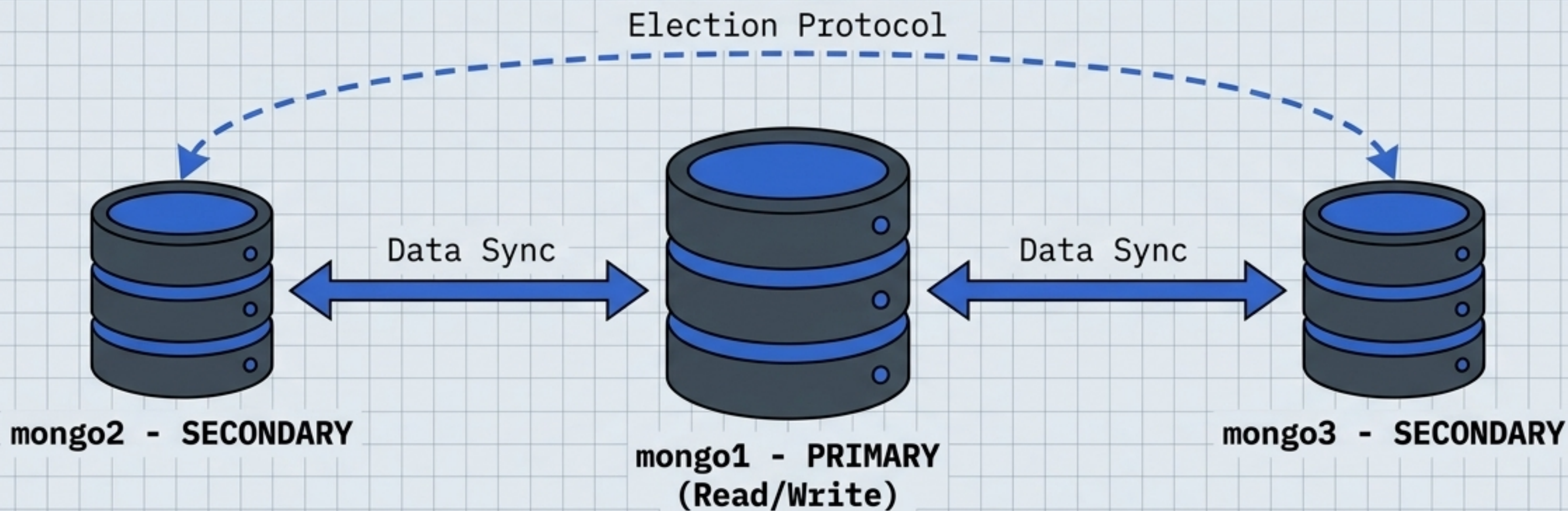
Mô hình Phi trạng thái (Stateless)

Các node xử lý bằng **Python** không lưu trạng thái nội bộ, cho phép mở rộng **scale-out** ngay lập tức khi tải hệ thống tăng cao.

Consumer Groups

Hỗ trợ xử lý song song các **partition** và tự động **rebalance** khối lượng công việc nếu phát hiện một node trong cụm bị sập.

Cơ chế MongoDB Replica Set



Data Redundancy

Dữ liệu có ít nhất 3 bản sao trên các mạng/container hoàn toàn riêng biệt. Các node Secondary liên tục đồng bộ dữ liệu từ Primary.

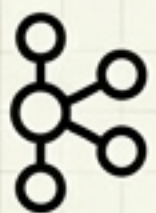
Auto-Failover Protocol

Nếu node Primary mất kết nối hoặc sập nguồn, quá trình bầu chọn (Election) giữa các node còn lại sẽ tự động kích hoạt ngay lập tức.

THỜI GIAN PHỤC HỒI

10 – 15 giây để lập ra Primary mới mà **không cần bất kỳ sự can thiệp thủ công nào.**

Cơ chế Chịu lỗi Toàn diện (Fault Tolerance)



Lớp Truyền dẫn (Kafka Replication)

Mỗi partition được nhân bản trên 2 broker. Nếu Leader sập, broker còn lại tự động **tiếp quản vai trò** Leader, đảm bảo luồng không đứt gãy.



Lớp Lưu trữ (MongoDB Replica Set)

Đồng bộ liên tục từ Primary sang 2 Secondary. Hệ thống chịu đựng được kích bản sập tối đa **1 node** mà vẫn giữ nguyên tính nhất quán.



Lớp Xử lý (Consumer Offset Management)

Cơ chế Commit Offset. Khi khởi động lại sau sự cố, node tiếp tục đọc từ offset cuối cùng được lưu trữ, cam kết không bỏ sót dữ liệu.



Lớp Triển khai (Docker & Physical Isolation)

Toàn bộ hệ thống được container **hóa** và phân bổ trên 4 máy tính vật lý. Sự cố phần cứng ở node xử lý được cô lập, không kéo sập node lưu trữ.

Kịch bản Thử nghiệm Chịu lỗi (Chaos Testing)

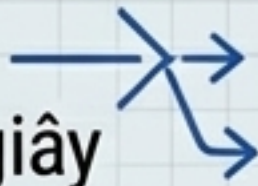
Hành động (Action)

Phản hồi Hệ thống (System Response)

 Tắt đột ngột node Consumer (Python Processing).	 Kafka lập tức kích hoạt rebalance. Các node Consumer còn lại tiếp quản phần việc trong vài giây. Dữ liệu tiếp tục được xử lý liên mạch.
 Tắt đột ngột node MongoDB Primary.	 Mất kết nối DB tạm thời. Quá trình bầu cử tự động (Election) diễn ra. Dashboard phục hồi và cập nhật dữ liệu hoàn toàn sau ~15 giây.

Kết luận: Kiến trúc phân tán đạt độ bền bỉ (**resilience**) cao, sẵn sàng cho các gián đoạn thực tế.

Đánh giá Hiệu năng Hệ thống

Hàng ngàn sự kiện / giây 

Khả năng Xử lý (Throughput)

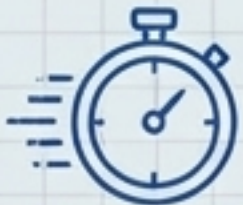
Chạy ổn định trên cấu hình mạng nội bộ (2 brokers + 5s **micro-batch**). Không nghẽn hàng đợi (**lag**), không mất mát dữ liệu.

5 - 10 giây

99.99% SLA 

Độ trễ Toàn trình (End-to-End Latency)

Bao gồm 5s chờ gom cửa sổ (tumbling window) + độ trễ truyền dẫn mạng + thời ghi MongoDB. Đạt tiêu chuẩn giám sát thực tế.

5 - 10 - 10 giây 

Chạy ổn định trên gan cửa sổ (tumbling window) + độ trễ truyền dẫn mạng + Không nghẽn **hàng đợi (lag)**, không **mất mát** dữ liệu.



99.99% SLA



Vượt qua toàn bộ các kịch bản **Chaos Testing**, cơ chế chuyển đổi dự phòng (**Failover**) diễn ra **mượt mà** và hoàn toàn **tự động**.

Đánh giá Ưu điểm và Hạn chế

Ưu điểm (+)

Sử dụng hoàn toàn **mã nguồn mở** (Kafka, Mongo, Docker), tối ưu chi phí.

Kiến trúc Microservices tách biệt, dễ dàng phát triển và mở rộng cục bộ.

Cơ chế chịu lỗi (Fault Tolerance) vững chắc ở cả lớp truyền dẫn và lưu trữ.

Dashboard theo dõi trực quan, tương tác thời gian thực.

Hạn chế (-)

Điểm nghẽn Python: Việc dùng Python thuần có thể không đáp ứng mức hàng triệu sự kiện/giây như các framework chuyên dụng (Flink/Spark).

Phụ thuộc nặng vào băng thông và chất lượng mạng nội bộ giữa các node.

Chưa áp dụng các phương thức bảo mật chuyên sâu cho dữ liệu truyền tải.

Trạng thái Hiện tại

Thành công cốt lõi

Hoàn thành 4 mục tiêu: Lưu trữ phân tán vững chắc, Thu thập luồng ổn định, Xử lý **micro-batch** tối ưu, và Cơ chế tự phục hồi tự động.

Mô hình đáp ứng tốt bài toán Big Data ở quy mô vừa.

Tương lai

Hướng Phát triển

Scale-out Architecture

Nâng cấp Xử lý: Thay thế Python thuần bằng Spark Streaming để tối đa hóa Throughput, hướng tới mục tiêu >10M sự kiện/giây.

Dịch chuyển Hạ tầng: Đưa toàn bộ cụm kiến trúc lên nền tảng Cloud (AWS/Azure) nhằm tận dụng năng lực Scale-out vô hạn.

Danh mục Từ viết tắt

API: Application Programming Interface

CDN: Content Delivery Network

HA: High Availability

SLA: Service Level Agreement

QoS: Quality of Service

RPC / HDFS: Remote Procedure Call / Hadoop Distributed File System

Tài liệu Tham khảo Cốt lõi

Brewer, E.A. (2000). Towards robust distributed systems (Định lý CAP).

Tanenbaum, A.S., Van Steen, M. (2017). Distributed Systems: Principles and Paradigms.

Kafka Engineering Team (2016). Netflix Keystone Pipeline.

Zaharia, M. et al. (2012). Resilient Distributed Datasets (Spark).